

Exercise set 2

Logistic regression and the K-nearest neighbors (KNN) classifier for a problem with two classes, zero and one

- In logistic regression (with one predictor), we model the conditional *probability* of $Y = 1$ given $X = x_0$.

$$P(Y = 1|X = x_0) = \frac{e^{\beta_0 + \beta_1 x_0}}{1 + e^{\beta_0 + \beta_1 x_0}} = \frac{1}{1 + e^{-\beta_0 - \beta_1 x_0}}.$$

- The KNN classifier estimates the conditional probability of $Y = 1$ given $X = x_0$ by

$$P(Y = 1|X = x_0) = \frac{1}{K} \sum_{x_i \in \mathbb{N}_0} I(y_i = 1)$$

where I is the indicator function and $\mathbb{N}_0$ are the K training-points that are closest to x_0 .

Task 2

Consider the dataset

x	y
1	0
2	1
3	0
4	1
5	1
6	1
7	1

- a) Fit a logistic regression of Y on X by

```
m1=glm(y~x,family="binomial")
summary(m1)
```

and write down the estimated equation for $P(Y = 1|X = x)$. Also, predict Y for $x_0 = 4$.

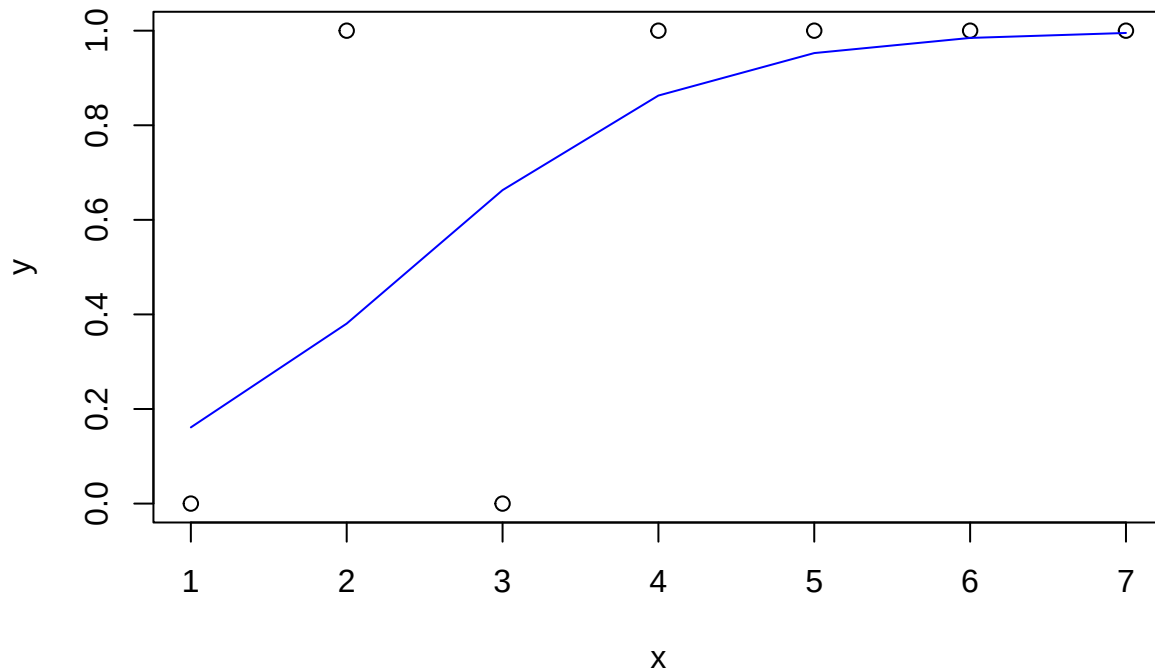
Solution The estimated equation is

$$\hat{P}(Y = 1|X = x) = \frac{1}{1 + \exp[2.813 - 1.163x]}$$

$\hat{P}(Y = 1|X = 4) = \frac{1}{1 + \exp[2.813 - 1.163 \cdot 4]} = 0.8628$. Using the threshold 0.5 we predict Y to be 1 if $x_0 = 4$.

- b) Compute the estimated probabilities and plot them on top of the observations by

```
plot(x,y)
m1=glm(y~x,family="binomial")
prob=predict(m1,type="response")
lines(x,prob,col="blue")
```



Which numbers do you get from the predict-function if you do not use the argument `type="response"`? Check by computing them yourself and by reading the help-function.

```
help(predict.glm)
```

Solution $\beta_0 + \beta_1 x$

```
head(predict(m1))
```

```
##           1           2           3           4           5           6
## -1.6497374 -0.4868367  0.6760640  1.8389647  3.0018654  4.1647661
```

```
head(-2.813+1.163*x) # Some rounding error
```

```
## [1] -1.650 -0.487  0.676  1.839  3.002  4.165
```

c) Use KNN with $K = 1, 3$ and 5 to estimate the conditional probability for $x_0 = 4$ by

```
knn=function(x0,x,y,K)
{
  d=abs(x0-x)
  o=order(d)
  prob=mean(y[o[1:K]])
  return(prob)
}
```

Explain each row of the code. Why can we interpret the result as a probability?

Solution

```
knn=function(x0,x,y,K) # Define a new function
{
  d=abs(x0-x) # Distance between x0 and all x-values
  o=order(d)  # The distances are ordered with the smallest
               # distance first. The first element of o
               # is the index corresponding to the smallest
               # distance
```

```

    prob=mean(y[o[1:K]]) # average of the y's corresponding
                        # to smallest x-distances
    return(prob)        # Return prob
}

```

In this special case the observations are zeros and ones. Imagine that we toss a coin and let zero be tails and one head. The number of heads divided by the number of tosses is then an estimated probability. This is the same as taking the average of all zeros and ones.

d) Plot estimated probabilities, using KNN for $x = 1, 2, \dots, 7$, on top of the observed data.

```

prob_knn=matrix(0,7,1)
for(i in 1:7) prob_knn[i]=knn(x0=x[i],x,y,K=3)
plot(x,y)
lines(x,prob_knn)

```

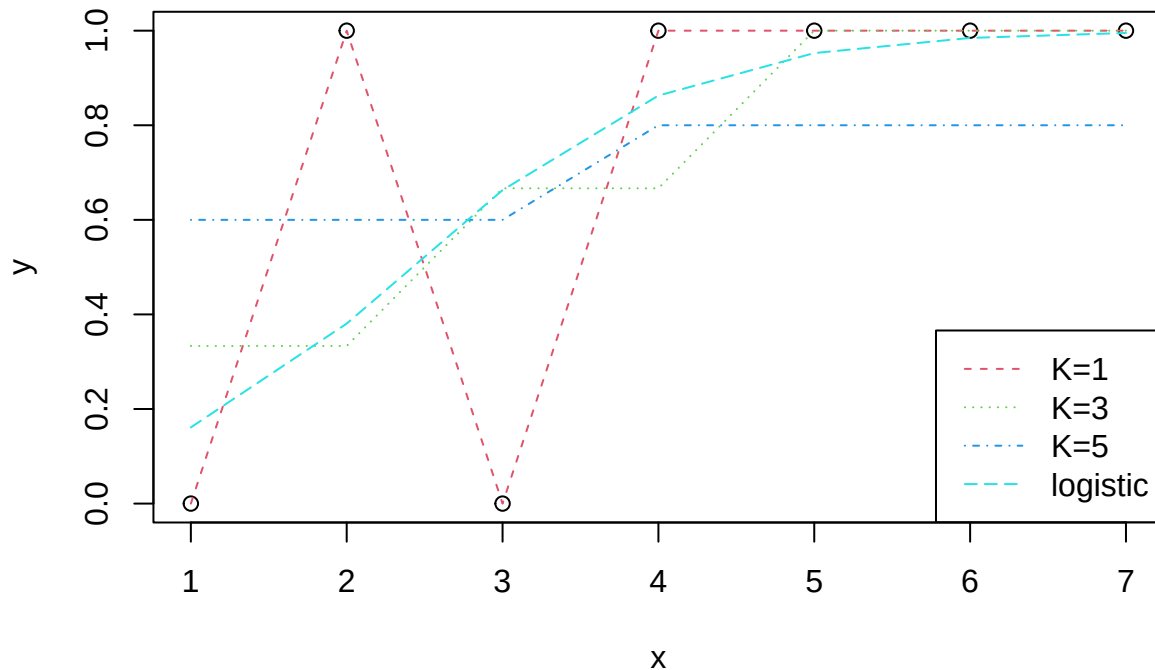
Can you add lines of estimated probabilities for $K = 1$, $K = 5$ and from logistic regression?

Solution

```

prob_knn1=matrix(0,7,1)
prob_knn3=matrix(0,7,1)
prob_knn5=matrix(0,7,1)
for(i in 1:7)
{
    prob_knn1[i]=knn(x0=x[i],x,y,K=1)
    prob_knn3[i]=knn(x0=x[i],x,y,K=3)
    prob_knn5[i]=knn(x0=x[i],x,y,K=5)
}
plot(x,y)
lines(x,prob_knn1,col=2,lty=2)
lines(x,prob_knn3,col=3,lty=3)
lines(x,prob_knn5,col=4,lty=4)
lines(x,prob,col=5,lty=5)
legend("bottomright",legend = c("K=1","K=3","K=5","logistic"),lty=2:5,col=2:5)

```



- e) Predict Y for $x = 1, 2, \dots, 7$, i.e., in the training data, by logistic regression and KNN and produce a confusion matrix. Use the estimated probabilities and convert them to predicted values (zero or one).
For logistic regression:

```
pred=prob>0.5
table(y,pred)
prop.table(table(y,pred),margin = 1)
```

To produce predictions from KNN, use e.g.,

```
pred_knn=prob_knn3>0.5
```

Solution

```
pred=prob>0.5
table(y,pred)
```

```
##      pred
## y  FALSE TRUE
## 0      1     1
## 1      1     4
```

```
prop.table(table(y,pred),margin = 1)
```

```
##      pred
## y  FALSE TRUE
## 0  0.5  0.5
## 1  0.2  0.8
```

```
pred_knn=prob_knn3>0.5
table(y,pred_knn)
```

```
##      pred_knn
## y  FALSE TRUE
## 0      1     1
## 1      1     4
```

```
prop.table(table(y,pred_knn),margin = 1)
```

```
##      pred_knn
## y    FALSE TRUE
## 0    0.5   0.5
## 1    0.2   0.8
```

The predictions are identical for this small dataset.

Linear discriminant analysis and logistic regression

- Discriminant analysis, in general, uses the principle of allocating (predicting) an observation i to the class k with the largest estimated $p_k(x_i) = P(Y = k|X = x_i)$ among all classes k
- Assume that we have two classes and the predictors, $\mathbf{X}$, in all classes, is normally distributed, with different expectation but the same variance (and covariances in the case of a multivariate $\mathbf{X}$). Then $p_1(x_i) > p_2(x_i)$ implies that $\delta_1(x_i) > \delta_2(x_i)$, where

$$\delta_k(x_i) = \ln \pi_k + \frac{\mu_k x_i}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2}$$

Since the *discriminant function* is a linear function of x_i , this approach is called *linear* discriminant analysis.

Task 3

Load the dataset [Default](#) in the R-package [ISLR](#) and do some descriptive data analysis to answer the following questions. The overarching aim is to predict the variable [default](#).

- Use both logistic regression and linear discriminant analysis to estimate the probability of default. Split the data 50/50 into a training and a test dataset.

```
library(ISLR)
n=nrow(Default)
ind=sample(1:n,size=floor(n/2))
train=Default[ind,]
test=Default[-ind,]
logreg=glm(default~.,data=train,family="binomial")
library(MASS)
lda=lda(default~balance+income,data=train)
```

Solution

```
library(ISLR)
n=nrow(Default)
ind=sample(1:n,size=floor(n/2))
train=Default[ind,]
test=Default[-ind,]
logreg=glm(default~.,data=train,family="binomial")
prob_logreg=predict(logreg,newdata=test,type="response")
library(MASS)
lda=lda(default~balance+income,data=train)
prob_lda=predict(lda,newdata=test)$posterior[,2]
head(prob_logreg)
```

```
##           2           3           4           7           8           9
## 0.0017233746 0.0119807569 0.0006719674 0.0030702769 0.0017023993 0.0196666683
```

```
head(prob_lda)
```

```
##           2           3           4           7           8           9
## 0.003570777 0.015081210 0.001402386 0.004499100 0.003735290 0.024156848
```

- b) Predict the test data using the estimated models. Compute confusion matrices. Do you get better predictions by removing some of the variables?

Solution

We predict 1 if the estimated probability is large than 0.5

```
pred_logreg=prob_logreg>0.5
pred_lda=prob_lda>0.5
prop.table(table(test$default,pred_logreg),margin=1)
```

```
##      pred_logreg
##      FALSE      TRUE
## No  0.997727273 0.002272727
## Yes 0.662500000 0.337500000
```

```
prop.table(table(test$default,pred_lda),margin=1)
```

```
##      pred_lda
##      FALSE      TRUE
## No  0.9991735537 0.0008264463
## Yes 0.7687500000 0.2312500000
```

The models are good at predicting $Y = 0$ correctly but not $Y = 1$. What if we change the threshold?

Cross-validation methods

- In the *validation set approach*, the dataset is split randomly in a training and a test dataset.
- In the *leave-one-out (LOO) approach*, the prediction model is fitted to a training data consisting of all but one observations and evaluated on the left-out observation. This leave-one-out is repeated for all observations.
- In the *k-fold cross-validation approach*, the observations are split into k parts where one part is used as the test data. This, leave-one-group-out, is then repeated for all groups.
- The term *test error* is here used, generically, for any evaluation of a prediction in the test data, e.g. testMSE or testER.

Task 4

- a) Consider the Auto dataset and create a class variable, `y`, which is “high” for observations where `mpg` is above its median and “low” when it is below. Convert it to a factor-variable. Compute age of the cars. Remove `mpg`, `name` and `year`.

```
library(ISLR)
Auto$y="low"
Auto$y[Auto$mpg>median(Auto$mpg)]="high"
Auto$y=as.factor(Auto$y)
Auto$age=83-Auto$year
Auto=Auto[,!(names(Auto) %in% c("mpg","name","year"))]
```

`mpg` is removed since it is substituted with `y`. `name` is removed since it contains too many unique values. `year` is removed since it is substituted with the linear transformation `age`. Explain each line of the code.

Solution

```

library(ISLR) # Load the ISLR library
Auto$y="low"  # Create a new variable y and set all
               # values to "low"
Auto$y[Auto$mpg>median(Auto$mpg)]= "high" # Set values larger than
                                             # the median to "high"
Auto$y=as.factor(Auto$y) # Recode y to be a factor
Auto$age=83-Auto$year     # Let age be the age of the car
Auto=Auto[,!(names(Auto) %in% c("mpg","name","year"))] # Remove
                                                         # variables

```

We will now use the validation set approach to investigate how well classification models, based on other variables in the dataset, predicts y .

- b) Split the data randomly into 50/50 training/test data, fit a logistic regression of y on all other variables and conclude which variables that you would like to try out as predictors.

Solution

```

n=nrow(Auto)
draw=sample(1:n,size=floor(n/2))
train=Auto[draw,]
test=Auto[-draw,]
m1=glm(y~.,data=train,family="binomial")
summary(m1)

##
## Call:
## glm(formula = y ~ ., family = "binomial", data = train)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.91589  -0.20415   0.00029   0.10203   2.51807
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -18.131555   5.525590  -3.281 0.001033 **
## cylinders    0.140737   0.597351   0.236 0.813742
## displacement -0.010685   0.017493  -0.611 0.541332
## horsepower   0.020975   0.035051   0.598 0.549572
## weight       0.006383   0.001908   3.344 0.000825 ***
## acceleration -0.234738   0.195333  -1.202 0.229468
## origin       -0.480059   0.500274  -0.960 0.337260
## age          0.548413   0.133292   4.114 3.88e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 271.632  on 195  degrees of freedom
## Residual deviance:  79.771  on 188  degrees of freedom
## AIC: 95.771
##
## Number of Fisher Scoring iterations: 8

```

Age and weight

- c) Predict y in the test data and compute a confusion matrix.

Solution

```
prob=predict(m1,newdata=test,type="response")
pred=prob>0.5
table(test$y,pred)
```

```
##      pred
##      FALSE TRUE
## high    91    9
## low     8    88
```

```
prop.table(table(test$y,pred),margin=1)
```

```
##      pred
##      FALSE      TRUE
## high 0.91000000 0.09000000
## low  0.08333333 0.91666667
```

d) Do the same thing with the leave-one-out approach. Make sure that you understand what the code is doing.

```
n=nrow(Auto)
pred=matrix(0,n,1)
for(i in 1:n)
{
  train=Auto[-i,]
  test=Auto[i,]
  m1=glm(y~.,data=train,family="binomial")
  prob=predict(m1,newdata=test,type="response")
  pred[i]=prob>0.5
}
prop.table(table(Auto$y,pred),margin=1)
```

```
##      pred
##      0      1
## high 0.92346939 0.07653061
## low  0.11734694 0.88265306
```

Solution

```
n=nrow(Auto) # Number of observations
pred=matrix(0,n,1) # n x 1 -vector
for(i in 1:n) # Go through all observations
{
  train=Auto[-i,] # Remove obs i to get the training data
  test=Auto[i,] # The i'th obs is the test data
  m1=glm(y~.,data=train,family="binomial") # Fit logistic reg
  prob=predict(m1,newdata=test,type="response") # Estimate prob of "low" on test
  pred[i]=prob>0.5 # Predict "low" if estimated prob > 0.5
}
prop.table(table(Auto$y,pred),margin=1)
```

```
##      pred
##      0      1
## high 0.92346939 0.07653061
## low  0.11734694 0.88265306
```

e) Do the same thing with 8-fold CV. Make sure that you understand what the code is doing.


```

n=nrow(Auto)
k=8
s=n/k # Size of each fold
pred=matrix(0,n,1)
ii=1
for(i in 1:k)
{
  train=Auto[-(ii:(ii+s-1)),]
  test=Auto[ii:(ii+s-1),]
  m1=glm(y~.,data=train,family="binomial")
  prob=predict(m1,newdata=test,type="response")
  pred[ii:(ii+s-1)]=prob>0.5
  ii=ii+s
}
prop.table(table(Auto$y,pred),margin=1)

```

```

##      pred
##           0           1
##  high 0.90306122 0.09693878
##  low  0.11734694 0.88265306

```

Solution

```

n=nrow(Auto) # Number of observations
k=8 # Number of folds
s=n/k # Size of each fold
pred=matrix(0,n,1) # n x 1 - vector
ii=1 # Index used to create the folds
for(i in 1:k)
{
  train=Auto[-(ii:(ii+s-1)),] # All but fold nr i
  test=Auto[ii:(ii+s-1),] # Fold nr i, e.g. observations 1 to s if # i=1
  m1=glm(y~.,data=train,family="binomial") # Fit logistic reg
  prob=predict(m1,newdata=test,type="response") # Estimate prob of "low" on test
  pred[ii:(ii+s-1)]=prob>0.5 # Predict "low" if estimated prob > 0.5
  ii=ii+s
}
prop.table(table(Auto$y,pred),margin=1)

```

```

##      pred
##           0           1
##  high 0.90306122 0.09693878
##  low  0.11734694 0.88265306

```

f) Would you prefer to use all predictors (input variables) or only some of them?

Solution

```

n=nrow(Auto)
pred=matrix(0,n,1)
for(i in 1:n)
{
  train=Auto[-i,]
  test=Auto[i,]
  m1=glm(y~age+weight,data=train,family="binomial")
  prob=predict(m1,newdata=test,type="response")
}

```

```
    pred[i]=prob>0.5
  }
prop.table(table(Auto$y,pred),margin=1)
```

```
##      pred
##           0         1
##  high 0.91836735 0.08163265
##  low  0.11224490 0.88775510
```

The results was worse when using age and weight only. Try out other variable combinations.